



JDK中的集合工具箱

北京理工大学计算机学院
金旭亮

JDK集合“工具箱”

JDK中提供了Arrays和Collections两个工具类，提供了诸多的与数组和集合相关的方法，其功能可以“望名知义”。

下面我们通过实例介绍这两个工具类中的一些常用方法.....



Collections工具类

JDK为数组提供的工具类称为Arrays，类似地，JDK中的集合工具类，称为Collections，封装了许多集合的常规操作，以下是开发中常用的部分集合方法的列表：

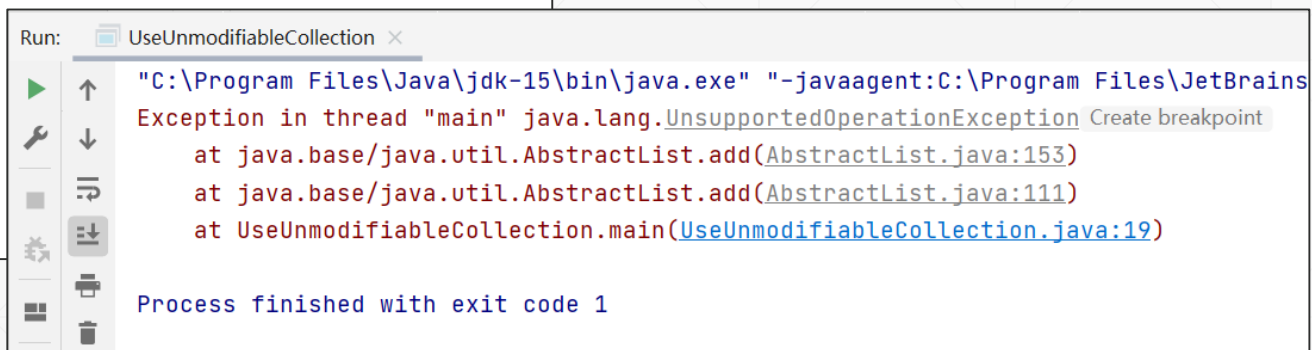
- 更改集合中元素的顺序：reverse、rotate、shuffle、sort和swap
- 用一个集合填充另一个集合：copy、fill和replaceAll、addAll
- 获取集合中的最大和最小元素：max、min
- 查找元素：binarySearch、indexOfSubList和lastIndexOfSubList
- 统计频数：frequency
- 判断交集：disjoint

Collections方法示例：创建只读集合

```
public class UseUnmodifiableCollection {  
    public static void main(String[] args) {  
        //创建一个空的、不可改变的List对象  
        List<String> unmodifiableList = Collections.emptyList();  
        //创建一个只有一个元素，且不可改变的Set对象  
        Set<String> unmodifiableSet = Collections.singleton("singleton");  
        //创建一个普通Map对象  
        Map<String, Integer> scores = new HashMap<>();  
        scores.put("语文", 80);  
        scores.put("Java", 82);  
        //返回普通Map对象对应的不可变版本  
        Map<String, Integer> unmodifiableMap =  
            Collections.unmodifiableMap(scores);  
        //下面任意一行代码都将引发UnsupportedOperationException异常  
        unmodifiableList.add("新元素");  
        unmodifiableSet.add("新元素");  
        unmodifiableMap.put("语文", 90);  
    }  
}
```

UseUnmodifiableCollection.java

只读集合可以安全地跨线程访问，
在开发中推荐使用。



```
Run: UseUnmodifiableCollection x  
"C:\Program Files\Java\jdk-15\bin\java.exe" "-javaagent:C:\Program Files\JetBrains  
Exception in thread "main" java.lang.UnsupportedOperationException Create breakpoint  
    at java.base/java.util.AbstractList.add(AbstractList.java:153)  
    at java.base/java.util.AbstractList.add(AbstractList.java:111)  
    at UseUnmodifiableCollection.main(UseUnmodifiableCollection.java:19)  
  
Process finished with exit code 1
```

Collections方法示例：创建同步集合

//下面程序代码创建了四个同步的集合对象

```
var collection : Collection<String> =  
    Collections.synchronizedCollection(new ArrayList<String>());  
var list : List<String> =  
    Collections.synchronizedList(new ArrayList<String>());  
var set : Set<String> =  
    Collections.synchronizedSet(new HashSet<String>());  
var map : Map<String, Integer> =  
    Collections.synchronizedMap(new HashMap<String, Integer>());
```

同步集合的特点是：它所提供的公有方法，在内部妥善处理了线程同步的问题，因此，跨线程访问它时，使用者不再需要对集合加锁，以保证多线程环境下数据存取正确。

使用同步集合示例-1

```
static void testSynchronizedList() throws InterruptedException {  
    //创建一个可以保存整数的同步集合  
    var list : List<Integer> = Collections.synchronizedList(  
        new ArrayList<Integer>());  
    Random ran = new Random();  
    //向集合中追加随机整数  
    Runnable addElementToList = new Runnable() {  
        @Override  
        public void run() {  
            for (int i = 0; i < 5; i++) {  
                int ranValue = ran.nextInt( bound: 100);  
                System.out.println(Thread.currentThread().getName()  
                    + "添加元素" + ranValue);  
                //add方法可以跨线程随意调用  
                list.add(ranValue);  
            }  
        }  
    };  
}
```

UseSynchronizedCollection.java

示例中创建了一个可以保存整数的同步List集合，然后，编写了两个Runnable匿名对象，这两个对象将被不同的线程执行。

```
//迭代访问List  
Runnable visitList = new Runnable() {  
    @Override  
    public void run() {  
        System.out.print("\n"+Thread.currentThread().getName()  
            + "输出List元素: ");  
        for (var num : list) {  
            System.out.print(num + ",");  
        }  
    }  
};
```

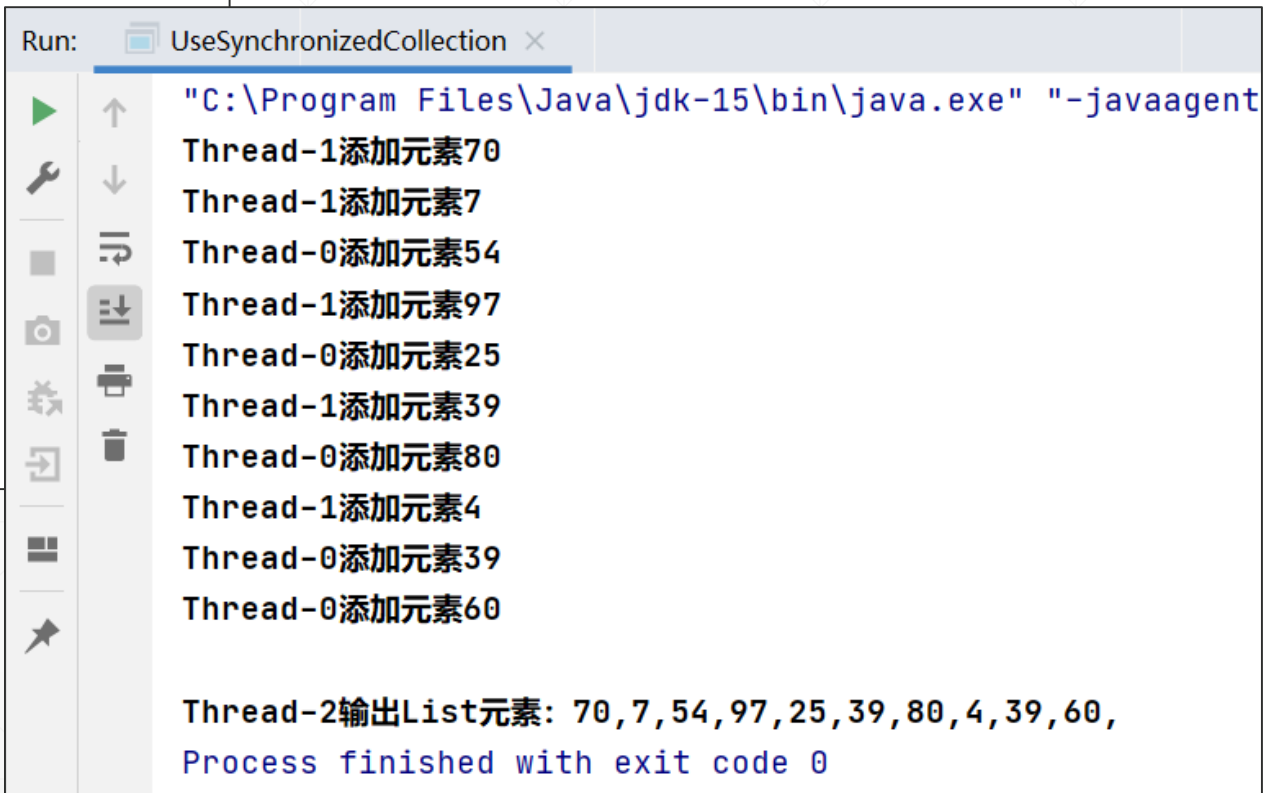

使用同步集合示例-2

```
//创建并启动两个线程，向List集合中添加随机整数
var addThread1 = new Thread(addElementToList);
var addThread2 = new Thread(addElementToList);
addThread1.start();
addThread2.start();

//等待元素添加完成
addThread1.join();
addThread2.join();

//当迭代访问同步集合时，仍然需要加锁
synchronized (list) {
    new Thread(visitList).start();
}
```

UseSynchronizedCollection.java



```
Run: UseSynchronizedCollection x
"C:\Program Files\Java\jdk-15\bin\java.exe" "-javaagent
Thread-1添加元素70
Thread-1添加元素7
Thread-0添加元素54
Thread-1添加元素97
Thread-0添加元素25
Thread-1添加元素39
Thread-0添加元素80
Thread-1添加元素4
Thread-0添加元素39
Thread-0添加元素60

Thread-2输出List元素: 70,7,54,97,25,39,80,4,39,60,
Process finished with exit code 0
```

同步集合能“同步”的奥秘

SynchronizedCollection<E>

```
/serial/ // Conditionally serializable
final Collection<E> c; // Backing Collection
/serial/ // Conditionally serializable
final Object mutex; // Object on which to synchronize

SynchronizedCollection(Collection<E> c) {
    this.c = Objects.requireNonNull(c);
    mutex = this;
}
```

SynchronizedList<E>

```
public void add(int index, E element) {
    synchronized (mutex) {list.add(index, element);}
}
```

从JDK中同步集合的源代码中可以看到，同步List类的方法（比如add）方法，实际上是在内部集成了“synchronized”代码块，从而保证这些代码一次只能被一个线程执行。

创建只读集合的of系列方法-1

JDK的各个集合类型，提供了一系列的of方法，可用于创建只读集合：

```
static void useListOf() {  
    List<String> list = List.of("One", "Two", "Three");  
    //输出: class java.util.ImmutableCollections$ListN  
    System.out.println(list.getClass());  
    //迭代访问  
    list.forEach(System.out::println);  
    //使用of系列方法创建的集合是只读的，创建完毕之后  
    //尝试向其添加元素，将抛出UnsupportedOperationException  
    //list.add("Four");  
}
```

UseOfMethods.java

创建只读集合的of系列方法-2

```
static void useSetOf() {  
    Set<String> set = Set.of("One", "Two", "Three");  
    //class java.util.ImmutableCollections$SetN  
    System.out.println(set.getClass());  
    //输出: Set is [Three, Two, One]  
    System.out.println("Set is " + set);  
}
```

UseOfMethods.java

创建只读集合的of系列方法-3

```
static void useMapOf() {  
    Map<String, LocalDate> specialDays = Map.of(  
        k1: "国庆节", LocalDate.of(year: 1949, Month.OCTOBER, dayOfMonth: 1),  
        k2: "五一节", LocalDate.of(year: 1886, Month.MAY, dayOfMonth: 1)  
    );  
    //class java.util.ImmutableCollections$MapN  
    System.out.println(specialDays.getClass());  
    specialDays.forEach((k, v) -> System.out.println(k + " on " + v));  
}
```

```
static void useMapOfEntries() {  
    Map<Integer, String> mapEntries = Map.ofEntries(  
        entry(k: 3, v: "Three"),  
        entry(k: 7, v: "Seven"),  
        entry(k: 1, v: "One")  
    );  
    //class java.util.ImmutableCollections$MapN  
    System.out.println(mapEntries.getClass());  
    mapEntries.forEach((k, v) -> System.out.println(k + " : " + v));  
}
```

可以使用两种of方法创建Map，其结果是一样的。

UseOfMethods.java

Collections集合元素常规操作

//原始数组

```
Character[] letters = {'P', 'C', 'M'};
```

//数组转为List

```
List<Character> list = Arrays.asList(letters);
```

```
System.out.println("转换之后: ");
```

```
output(list);
```

// 反转List

```
Collections.reverse(list);
```

```
System.out.println("\n反转之后: ");
```

```
output(list);
```

ProcessList.java

// 复制List

```
List<Character> copyList =
```

```
    Arrays.asList(new Character[3]);
```

```
Collections.copy(copyList, list);
```

```
System.out.println("\n复制之后: ");
```

```
output(copyList);
```

// 用特定的元素填充List

```
Collections.fill(list, obj: 'R');
```

```
System.out.println("\n填充之后: ");
```

```
output(list);
```

基于集合的操作-1

//创建两个颜色集合

```
String[] colors = {"red", "white", "yellow", "blue"};  
List<String> list1 = Arrays.asList(colors);  
ArrayList<String> list2 = new ArrayList<>();  
list2.add("black");  
list2.add("red");  
list2.add("green");
```

使用addAll，可以合并数组
(或多个类型兼容的对象)
到集合中。

ListOperation.java

```
System.out.print("合并前, list2包容: ");  
System.out.println(list2);  
//将colors数组合并到list2集合  
Collections.addAll(list2, colors);  
System.out.print("\n合并后, list2包容: ");  
System.out.println(list2);
```

基于集合的操作-2

```
// 获取单词 "red"的出现频数
int frequency = Collections.frequency(list2, o: "red");
System.out.printf(
    "\nlist2中red出现次数: %d\n", frequency);

// 两个集合中是否包容有共同的元素
boolean disjoint = Collections.disjoint(list1, list2);

System.out.printf("list1和list2包容相同的元素? " + disjoint);
```

ListOperation.java

集合排序

```
public class CollectionSort {  
    public static void main(String[] args) {  
        String[] suits = {"Hearts", "Diamonds", "Clubs", "Spades"};  
        //将数组转换为集合  
        List<String> list = Arrays.asList(suits);  
        System.out.printf("原始未排序的集合: %s\n", list);  
        //升序排序  
        Collections.sort(list);  
        //输出集合内容  
        System.out.printf("升序排序之后: %s\n", list);  
        // 降序排列, 使用Comparator  
        Collections.sort(list, Collections.reverseOrder());  
        System.out.printf("降序排序之后: %s\n", list);  
    }  
}
```

编程练习：随机打乱集合中的元素



使用`Collections.shuffle()`方法，可以打乱集合中元素的顺序。

请编写一个示例，用`shuffle()`方法模拟“洗扑克牌”的功能。

参考解答：`DeckOfCards.java`



搜索

//使用二分法在已排好序的集合中查找元素

```
private static void searchElement(  
    List<String> list, String element) {  
    int result = 0;  
    System.out.printf("\n二分查找: %s\n", element);  
    result = Collections.binarySearch(list, element);  
    if (result >= 0)  
        System.out.printf("要查找的元素位置索引为: %d\n", result);  
    else  
        System.out.printf("没有找到 (%d)\n", result);  
}
```

BinarySearchDemo.java

小结



- JDK中提供了多种多样的集合组件和功能相当丰富的工具类
- 在实际开发中，要注意不同种类的集合适用于不同的场景，注意避开有些集合所包容的“陷阱”。
- 另外还要注意不要重复发明轮子，JDK中已经实现的功能，就直接拿过来用就好了。
- 推荐有时间时，仔细地读一读JDK中的各种集合与工具类型的源代码，有助于提升你的编程能力。